

Working with Abstraction

Architecture, Refactoring and Enumerations



SoftUni Team
Technical Trainers



**Software
University**



**SoftUni
Foundation**



Software University

<http://softuni.bg>

1. Project Architecture

- Methods
- Classes
- Projects

2. Code Refactoring

3. Enumerations



sli.do

#fund-csharp



Project Architecture

Splitting Code into Logical Parts

Splitting Code into Methods

- We use **methods** to split code into functional blocks
 - Improves code **readability**
 - Allows for easier **debugging**

```
foreach (char move in moves)
{
    for (int r = 0; r < room.Length; r++)
        for (int c = 0; c < room[r].Length; c++)
            if (room[r][c] == 'b')
                ...
}
```



```
foreach (char m in moves)
{
    MoveEnemies();
    KillerCheck();
    MovePlayer(move);
}
```

Splitting Code into Methods (2)

- **Methods** let us easily **reuse** code
- We change the **method** once to affect **all calls**

```
var bankAcc = new BankAccount();  
bankAcc.Id = 1;  
bankAcc.Deposit(20);  
Console.WriteLine($"Account {bankAcc.Id}, balance {bankAcc.Balance}");  
bankAcc.Withdraw(10);  
...  
Console.WriteLine($"Account {bankAcc.Id}, balance {bankAcc.Balance}");
```

Override `.ToString()` to set
a global printing format

↳ `Console.WriteLine(bankAcc.ToString());`

Splitting Code into Methods (3)

- A **single** method should complete a **single task**

```
void DoMagic ( ... )  
void DepositOrWithdraw ( ... )  
decimal DepositAndGetBalance ( ... )  
string ParseDataAndReturnResult ( ... )
```



```
void Withdraw ( ... )  
void Deposit ( ... )  
decimal GetBalance ( ... )  
string ToString ( ... )
```



Problem: Rhombus of Stars

- Draw on the console a rhombus of stars with size **n**

$n = 3$



```
*  
* *  
* * *  
* *  
*
```

$n = 2$

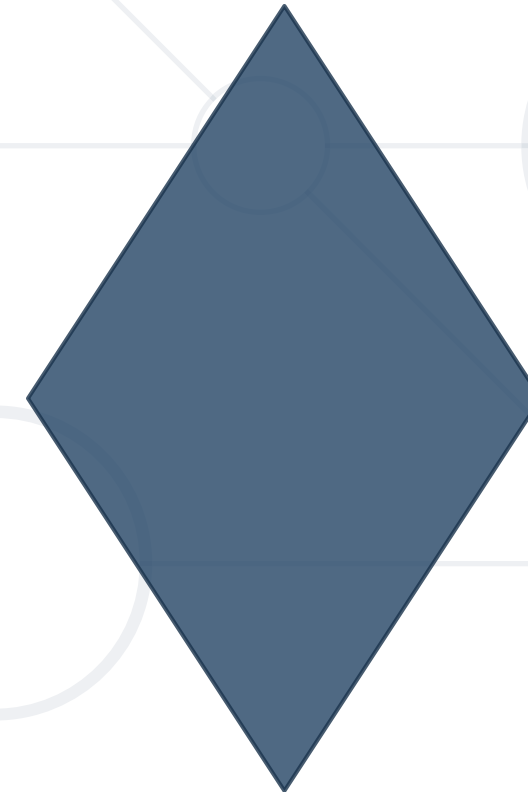


```
*  
* *  
*
```

$n = 1$



```
*
```



Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

Solution: Rhombus of Stars

```
int size = int.Parse(Console.ReadLine());  
for (int starCount = 1; starCount <= size; starCount++)  
{  
    PrintRow(size, starCount);  
}  
for (int starCount = size - 1; starCount >= 1; starCount--)  
{  
    PrintRow(size, starCount);  
}
```

Reusing code

Solution: Rhombus of Stars(2)

```
static void PrintRow(int figureSize, int starCount)
{
    for (int i = 0; i < figureSize - starCount; i++)
        Console.Write(" ");
    for (int col = 1; col < starCount; col++)
        Console.Write("* ");
    Console.WriteLine("*");
}
```

Splitting Code into Classes

- Just like methods, **classes** should **not** know or do too much



```
GodMode master = new GodMode();  
int[] numbers = master.ParseAny(args);  
...  
int[] numbers2 = master.CopyAny(numbers);  
master.PrintToConsole(master.GetDate());  
master.PrintToConsole(numbers);
```

Splitting Code into Classes (2)

- We can also break our code up logically into **classes**

```
int[] numbers = args.Split().Select(int.Parse).ToArray();  
Console.WriteLine(string.Join(" ", numbers));
```



Hiding implementation

```
ArrayParser parser = new ArrayParser();  
OutputWriter printer = new OutputWriter();  
int[] numbers = parser.IntegersParse(args);  
int[] coordinates = parser.IntegerParse(args1);  
printer.PrintToConsole(numbers);
```

Allow us to change
output destination

Helps us avoid
repeating code

Problem: Point in Rectangle

- Create a Point class holding the horizontal and vertical coordinates
- Create a **Rectangle class**
 - Holds 2 **points**
 - **Top left** and **bottom right**
- Add **Contains** method
 - Takes a **Point** as an argument
 - **Returns** it if it's inside the current object of the **Rectangle class**

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

Solution: Point in Rectangle

```
public class Point
{
    private int x;
    private int y;
    //TODO: Add Public properties
}

public class Rectangle
{
    private Point topLeft;
    private Point bottomRight;
    //TODO: Public properties
    public bool Contains(Point point)
    { //TODO: Implement }
}
```

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

Solution: Point in Rectangle (2)

```
public bool Contains(Point point)
{
    bool isInHorizontal =
        this.TopLeft.X <= point.X &&
        this.BottomRight.X >= point.X;

    bool isInVertical =
        this.TopLeft.Y <= point.Y &&
        this.BottomRight.Y >= point.Y;

    bool isInRectangle = isInHorizontal && isInVertical

    return isInRectangle;
}
```

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>



Refactoring

Restructuring and Organizing Code

Refactoring

- **Restructures** code without changing the behaviour
- **Improves** code readability
- **Reduces** complexity



```
class ProblemSolver { public static void DoMagic() { ... } }
```



```
class CommandParser { public static Func<T, T> ParseCommand(); }  
class DataModifier { public static T Execute(); ... }  
class OutputFormatter { public static void Print(); ... }
```

- **Breaking code** into reusable units
- **Extracting parts of methods** and **classes** into **new** ones

```
DepositOrWithdraw()
```



```
Deposit()  
Withdraw()
```

- **Improving names** of variables, methods, classes, etc.

```
string str;
```



```
string name;
```

- **Moving methods** or **fields** to more appropriate classes

```
Car.Open()
```

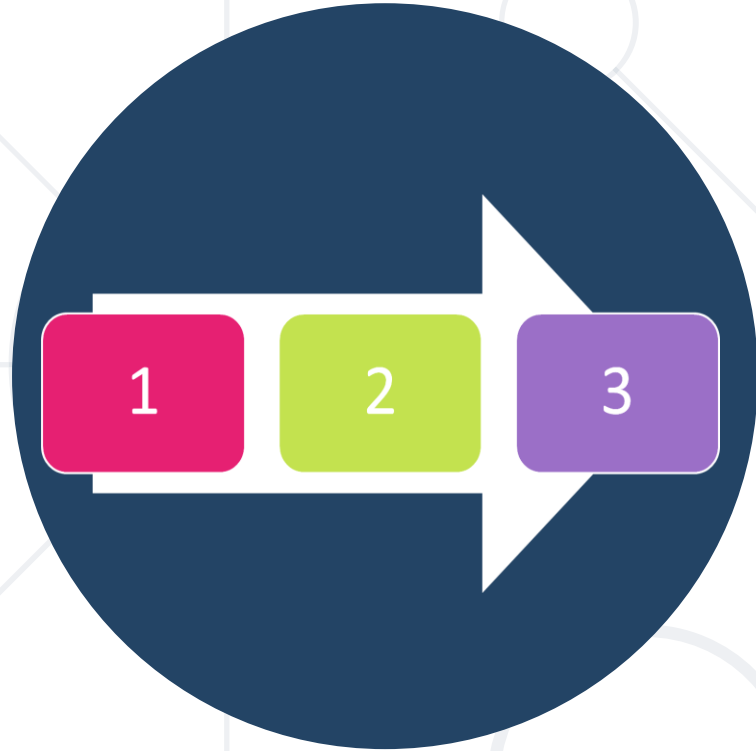


```
Door.Open()
```

Problem: Student System

- You are given a **working** Student System project to refactor
- **Break it up** into smaller functional units and make sure it works
- It supports the following **commands**:
 - "Create <studentName> <studentAge> <studentGrade>"
 - creates a new student
 - "Show <studentName>"
 - prints information about a student
 - "Exit"
 - closes the program

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>



Enumerations

Syntax and Usage

Enumerations

- Represent a numeric value from a fixed set as a text
- We can use them to pass **arguments** to **methods** without making code confusing

```
enum Day { Mon, Tue, Wed, Thu, Fri, Sat, Sun }
```

```
GetDailySchedule(0)
```



```
GetDailySchedule(Day.Mon)
```

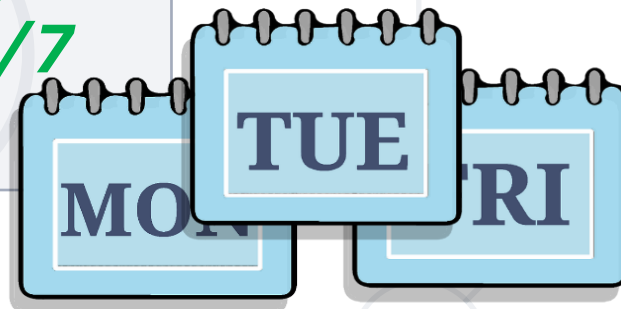
- By default **enums** start at 0
- Every next value is incremented by 1



Enumerations (2)

- We can **customize** enum **values**

```
enum Day
{
    Mon = 1,
    Tue,      //2
    Wed,      //3
    Thu,      //4
    Fri,      //5
    Sat,      //6
    Sun       //7
}
```



```
enum CoffeeSize
{
    Small = 100,
    Normal = 150,
    Double = 300
}
```



Problem: Hotel Reservation

- Create a class PriceCalculator that calculates the total price of a holiday, by given **price per day**, **number of days**, **the season** and a **discount type**
- The discount type and season should be **enums**
- The price multipliers will be:
 - 1x for Autumn, 2x for Spring, etc.
- The discount types will be:
 - None – 0%
 - SecondVisit – 10%
 - VIP – 20%



Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

Solution: Hotel Reservation

```
public enum Season
{
    Spring = 2,
    Summer = 4,
    Autumn = 1,
    Winter = 3
}

public enum Discount
{
    None,
    SecondVisit = 10,
    VIP = 20
}
```

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

Solution: Hotel Reservation (2)

```
public class PriceCalculator
{
    public static decimal CalculatePrice(decimal pricePerDay, int
        numberOfDays, Season season, Discount discount)
    {
        int multiplier = (int)season;
        decimal discountMultiplier = (decimal)discount / 100;

        decimal priceBeforeDiscount = numberOfDays * pricePerDay * multiplier;
        decimal discountedAmount = priceBeforeDiscount * discountMultiplier;
        decimal finalPrice = priceBeforeDiscount - discountedAmount;

        return finalPrice;
    }
}
```

Check your solution here: <https://judge.softuni.bg/Contests/955/Working-with-Abstraction-Lab>

- Well organized code is easier to work with
- We can reduce complexity using **Methods, Classes** and **Projects**
- We can refactor existing code by **breaking code apart**
- **Enumerations** define a fixed **set of constants**
 - Represent **numeric values**
 - We can easily **cast enums** to **numeric types**



SoftUni Diamond Partners



XSsoftware



SBTech
we know sports



telenor



SoftwareGroup
doing it right

NETPEAK



SmartIT



Postbank

Решения за твоето утре

**SUPER
HOSTING**
.BG

INDEAVR

Serving the high achievers



INFRAGISTICS®

LIEBHERR



æternity



codexio

SoftUni Organizational Partners



OneBit
SOFTWARE



Trainings @ Software University (SoftUni)

- Software University – High-Quality Education and Employment Opportunities
 - softuni.bg
- Software University Foundation
 - <http://softuni.foundation/>
- Software University @ Facebook
 - facebook.com/SoftwareUniversity
- Software University Forums
 - forum.softuni.bg



- This course (slides, examples, demos, videos, homework, etc.) is licensed under the "Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International" license

